

SNORT RESEARCH AND ANALYSIS

Snort is a widely used open-source network IDS and IPS that analyzes IP traffic in real time. It was created by Martin Roesch in 1998, originally for Unix-like systems. Roesch also produced SourceFire which commercialized Snort. Snort was acquired by Cisco in 2013 for \$2.7 billion and is now their primary developer. It was introduced as a simple packet sniffer but is now used by over 600,000 users with 5 million + downloads. Snort is constantly being improved as new versions release like Snort 3 in 2021. Snort contains many features such as packet sniffing, NIDS/NIPS, and packet logging. Snort provides real time traffic analysis with DPI, rule-based detection by reading signatures and anomalous behavior, and protocol analysis to find deeper malicious content embedded in packets. Cisco primarily maintains and develops Snort. Cisco's Talos Intelligence Group manages and approves the community ruleset as it is an open-source project with global contributors. Talos also develops and approves their official Snort Subscriber Ruleset.

Snort works as a NIDS/NIPS and is the industry standard for network-based detection. It is a great resource for cybersecurity cases as it is heavy on attack detection for malicious attacks like SQL injection, forged IP packets, and buffer overflows. Snort can work as NIPS by blocking malicious packets or resetting suspicious connections. Snort can track port scanning or other reconnaissance probing for open services within a network. Snort enforces the protocol usage of a packet by flagging suspicious packets that can help prevent zero day attacks. Security analysts in SOC analyze Snort's logs to reconstruct an intrusion in order to discover how the attack occurred. Snort paired with SIEM tools such as Splunk or Wazuh can work together to present security events across an organization. Snort can adjust to written rules to monitor against new attacks and vulnerabilities before a vendor patch is available.

Snort acts as an automated inspection tool that monitors packets entering or leaving the network interface. The data is processed through 4 components. The packet decoder inspects the protocols that are used and ensures the packet isn't forged or corrupted before entering the preprocessor. The preprocessor analyzes the packet for encoding or hidden attacks that may be fragmented. The packet is reassembled into the correct order so the detection system can register any legitimate or illegitimate traffic. The detection engine then compares the packets against Snort rules to common signatures in legitimate traffic. Snort primarily used signature-based systems to look for

a script or suspicious connection attempt. If the packet matches a rule, an alert is triggered or the packet is dropped. The output module makes the decision of what to do with the information. An alert can be sent to the database in real-time, the event can be logged, or it can be integrated with a SIEM tool. This ensures the network admin can view the alert.

Snort is primarily a signature-based NIDS/NIPS, following predetermined rules. There are 2 components of a rule: the header and options. The header specifies the action to occur like an alert or log. The header also includes the source and destination IP address, source and destination port number, and protocol. The option is where a security admin can see what is in the packet and if a packet matches one of the rules to give off an alert. A simple example of a snort rule is preventing an email server at a specific IP address. To prevent a specific IP subnet from using the server we can block any content containing a specific string. Mail servers that work on SMTP use port 25 and to block a string like "hacking" we can use the content keyword. "alert tcp 192.168.1.0/24 any -> 131.171.127.1 25 (content: "hacking"; msg: "malicious packet"; sid:2000001;)" This Snort rule prevents any IP address in 192.168.1.0/24 subnet from using port 25 on the email server IP address of 131.171.127.1 and the alert received is a "malicious packet".

Snort has high versatility in network security. Snort's packet sniffer mode reads packets off the network and works like a tcpdump. This is best for quick troubleshooting and ensuring that the network interface is seeing traffic. The content is then placed on the console. In the packet logger mode, Snort will log all IP packets that access the network. A network admin can view the users that access the network and the different protocols or OS that they are using. The packets recorded are placed right on the hard drive and can be viewed using tools like WireShark for analysis. Packets can be logged in plaintext or binary format. This is best for forensic analysis or having logs to update the signature rules. The NIDPS mode only logs packets that are malicious. This uses Snort's rule based system previously explained to define the action that the network admin will take. Network traffic is actively monitored and used for threat prevention. This mode allows one to drop the traffic or it can be configured to send an alert.

Snort can be used to detect and send alerts. A worm called conflicker A was found in 2008. Using the shellcode and binary string that is found online, you can craft a rule that sends an alert to prevent the worm from infiltrating port 445. After, an alert should be sent to the security admin. Some common limitations of Snort are false positives, performance bottlenecks, rule limitations, and network latency due to high volume of traffic. Encrypted SSL/TLS packets may prevent packet inspection of payloads without a separate decryption solution.

Running Snort 3 on Ubuntu

```
skarim@ubuntu: ~/Desktop — sudo snort -c /...
skarim@ubuntu: ~/Desktop — sudo snort -c /...
total: 219 219
-----
fast pattern groups
  to_server: 1
  to_client: 1
-----
search engine (ac_bnf)
  instances: 2
  patterns: 438
  pattern chars: 2602
  num states: 1832
  num match states: 392
  memory scale: KB
  total memory: 71.2812
  pattern memory: 19.6484
  match list memory: 28.4375
  transition memory: 22.9453
appid: MaxRss diff: 4524
appid: patterns loaded: 300
-----
pcap DAQ configured to passive.
Commencing packet processing
Retry queue interval is: 200 ms
++ [0] enp0s1
```

Included written rules into Snort config

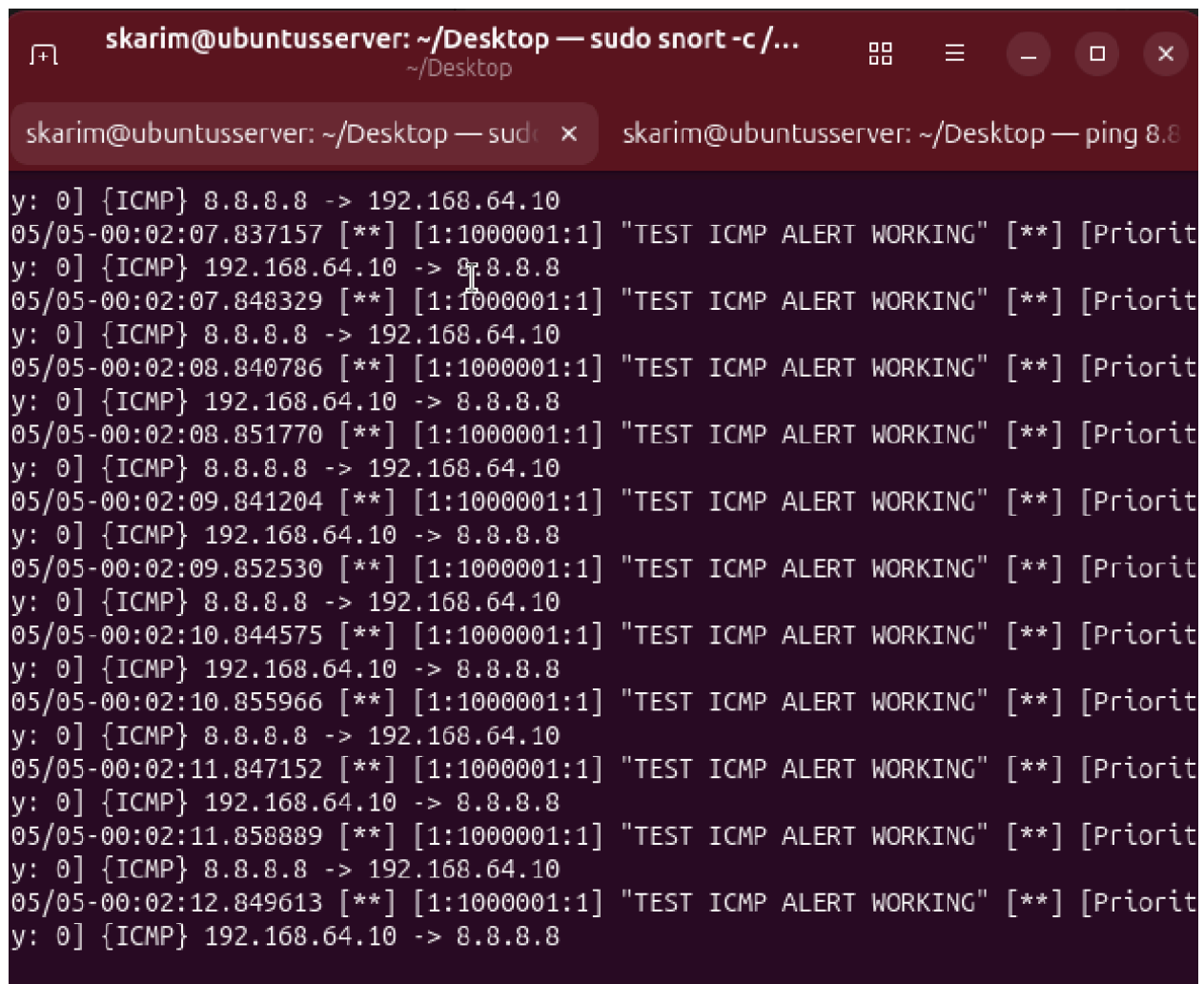
```
ips =
{

rules = [[
  include /usr/local/etc/snort/rules/local.rules
]],
# use this to enable decoder and inspector alerts
```

Writing rule into local.rules file

```
skarim@ubuntu: ~/Desktop — sudo nano /usr...
skarim@ubuntu: ~/Desktop — sudo snort -c /...
skarim@ubuntu: ~/Desktop — sudo snort -c /...
GNU nano 8.7.1 /usr/local/etc/snort/rules/local.rules *
alert icmp any any -> any any (msg:"TEST ICMP ALERT WORKING"; sid:1000001; rev:1)
```

ICMP alerts from IDS mode

A screenshot of a terminal window on a Ubuntu server. The window title is "skarim@ubuntusserver: ~/Desktop — sudo snort -c /...". The terminal shows a series of ICMP alerts generated by Snort. Each alert line follows the format: "y: 0] {ICMP} [source IP] -> [destination IP] [timestamp] [confidence] [rule ID] [rule signature] [confidence] [priority]". The alerts show traffic between 192.168.64.10 and 8.8.8.8. The rule signature for all alerts is "TEST ICMP ALERT WORKING". The terminal output is as follows:

```
y: 0] {ICMP} 8.8.8.8 -> 192.168.64.10
05/05-00:02:07.837157 [**] [1:1000001:1] "TEST ICMP ALERT WORKING" [**] [Priority
y: 0] {ICMP} 192.168.64.10 -> 8.8.8.8
05/05-00:02:07.848329 [**] [1:1000001:1] "TEST ICMP ALERT WORKING" [**] [Priority
y: 0] {ICMP} 8.8.8.8 -> 192.168.64.10
05/05-00:02:08.840786 [**] [1:1000001:1] "TEST ICMP ALERT WORKING" [**] [Priority
y: 0] {ICMP} 192.168.64.10 -> 8.8.8.8
05/05-00:02:08.851770 [**] [1:1000001:1] "TEST ICMP ALERT WORKING" [**] [Priority
y: 0] {ICMP} 8.8.8.8 -> 192.168.64.10
05/05-00:02:09.841204 [**] [1:1000001:1] "TEST ICMP ALERT WORKING" [**] [Priority
y: 0] {ICMP} 192.168.64.10 -> 8.8.8.8
05/05-00:02:09.852530 [**] [1:1000001:1] "TEST ICMP ALERT WORKING" [**] [Priority
y: 0] {ICMP} 8.8.8.8 -> 192.168.64.10
05/05-00:02:10.844575 [**] [1:1000001:1] "TEST ICMP ALERT WORKING" [**] [Priority
y: 0] {ICMP} 192.168.64.10 -> 8.8.8.8
05/05-00:02:10.855966 [**] [1:1000001:1] "TEST ICMP ALERT WORKING" [**] [Priority
y: 0] {ICMP} 8.8.8.8 -> 192.168.64.10
05/05-00:02:11.847152 [**] [1:1000001:1] "TEST ICMP ALERT WORKING" [**] [Priority
y: 0] {ICMP} 192.168.64.10 -> 8.8.8.8
05/05-00:02:11.858889 [**] [1:1000001:1] "TEST ICMP ALERT WORKING" [**] [Priority
y: 0] {ICMP} 8.8.8.8 -> 192.168.64.10
05/05-00:02:12.849613 [**] [1:1000001:1] "TEST ICMP ALERT WORKING" [**] [Priority
y: 0] {ICMP} 192.168.64.10 -> 8.8.8.8
```

I downloaded the Snort 3.0 packages on Ubuntu and wrote a rule "alert icmp any any -> any any (msg:"TEST ICMP ALERT WORKING"; sid:1000001; rev:1;)" to receive alerts on any pings for testing on IDS mode. I also included all rules in the I was able to ping an external IP address 8.8.8.8 from my own device and use a loopback interface. The console then displayed all live alerts in IDS mode.

Part 3 Review:

Snort can be used as an NIDS/NIPS system to capture network traffic and analyze it. I can use it to configure alerts for any type of packet based on rules implemented into the

local.rules. Depending on what type of traffic I want to permit, I can tell Snort to send an alert or drop the packet itself. This can prevent many types of attacks that would seep through the firewall and stop it from ever entering the internal network. I would prefer to use a signature-based system rather than an anomaly based system because I can directly allow what traffic can access the network. Despite being susceptible to false negatives, I think that it would be a strong asset in my network alongside a layered defense. As I am monitoring the systems, I will update every known attack into the database of threats to prevent any malicious attacks.

Sources:

Lenaerts-Bergmans, B. (2023, April 24). *Snort Explained: Understanding Snort Rules and Use Cases* | CrowdStrike. Crowdstrike.com.

<https://www.crowdstrike.com/en-us/cybersecurity-101/threat-intelligence/snort-rules/>

Snort. (2019). *Snort - Network Intrusion Detection & Prevention System*.

Snort.org; Snort. <https://www.snort.org/>

What is Snort? Bonus: How to Write Snort Rules! (2018, March 19). The Cybersecurity Man.

<https://thecybersecurityman.com/2018/03/19/what-is-snort-bonus-how-to-write-snort-rules/>